

Architecture de déploiement dissocié

Recette (branche `staging` → Contabo / horus-lab) & Cliente (branche `main` → serveur DMZ). Mis à jour le 30/07/2026.

1. Principe — deux environnements, deux branches

`staging` → images `:staging` (domaines **horus-lab**) → **Contabo recette**
`main` → images `:main` (domaines **client**) → **Serveur cliente (DMZ)**

Branche	Serveur	Tag d'images	Domaines figés au build	Workflow de déploiement
<code>staging</code>	Contabo recette (<code>gathe-finance.horus-lab.com</code>)	<code>:staging</code>	*.horus-lab.com	<code>deploy-staging.yml</code> (nginx-external, secrets <code>VPS_*</code>)
<code>main</code>	Serveur cliente (DMZ Traefik)	<code>:main</code>	*.gathe-finance.com	<code>deploy.yml</code> (rattachement DMZ, secrets <code>CLIENT_*</code>)

2. Le point technique à comprendre (le piège)

La CI tague les images produites selon la branche (via `docker/metadata-action`) :

- Push **staging** → `:staging` + `:sha-xxxx` — **domaines horus-lab** figés dans le front (NEXT_PUBLIC_*).
- Push **main** → `:main` + `:latest` + `:sha-xxxx` — **domaines client** figés dans le front.

⚠ Conséquence : historiquement, horus-lab tirait `:latest`. Or `:latest` n'est produit que par **main**. Le jour où `main` devient « cliente », `:latest` porte les **domaines client** → la recette se casserait en tirant `:latest`. **Donc la recette doit tirer `:staging`** (variable serveur `GATHE_IMAGE_TAG=staging`).

✓ **Règle d'or** : horus-lab ⇒ `GATHE_IMAGE_TAG=staging` · serveur cliente ⇒ `GATHE_IMAGE_TAG=main` .

3. Procédure de mise en place (ordre impératif)

1. Créer `deploy-staging.yml` — déclenché sur CI verte de `staging`, déploie horus-lab (`docker-compose.prod.yml` + `docker-compose.nginx-external.yml`, secrets `VPS_*`), tag `:staging` .
2. Poser `deploy-staging.yml` sur **main** . Les déclencheurs `workflow_run` ne s'exécutent QUE depuis la branche par défaut (`main`) — sinon le workflow ne se lance jamais.
3. Basculer horus-lab sur `:staging` — commande §4.A (avant que `main` ne produise du `:latest` client).
4. Merger le travail → `staging` → CI build `:staging` → `deploy-staging.yml` → **recette en ligne**.
5. **main** reste cliente : `deploy.yml` se déclenche mais le garde-fou **s'ignore** tant que le secret `CLIENT_VPS_HOST` est vide. Aucun déploiement accidentel.

4. Commandes serveur

A. Recette horus-lab — bascule sur `:staging` (1 fois)

```
ssh root@81.0.246.144 # clé de déploiement (gathe_ci)
cd /opt/gathe-finance/infra
sed -i 's/^GATHE_IMAGE_TAG=.*GATHE_IMAGE_TAG=staging/' .env.prod
grep GATHE_IMAGE_TAG .env.prod # doit afficher =staging
COMPOSE="docker compose -f docker-compose.prod.yml -f docker-compose.nginx-external.yml --env-file .env.prod"
$COMPOSE pull
$COMPOSE up -d
$COMPOSE ps # backend healthy attendu
```

B. Serveur cliente — rendre le déploiement `main` effectif (bootstrap DMZ)

À faire une seule fois. Détail complet : `docs/RUNBOOK_DEPLOIEMENT_CLIENT_DMZ.pdf` + `infra/scripts/inspect-dmz.sh`.

```
# 1) Relevé de la DMZ (réseaux, certresolver, Postgres, MinIO) – lecture seule
bash /opt/gathe-finance/infra/scripts/inspect-dmz.sh

# 2) Base Postgres dédiée (dans LEUR Postgres, réseau data)
psql -U postgres -c "CREATE DATABASE gathe_prod;"
psql -U postgres -c "CREATE USER gathe WITH PASSWORD '*****';"
psql -U postgres -c "GRANT ALL PRIVILEGES ON DATABASE gathe_prod TO gathe;"

# 3) Bucket MinIO + clés d'accès
mc mb local/gathe-media
mc admin user svcacct add local <compte> # -> access key + secret

# 4) infra/.env.prod (copie de .env.prod.client.example) – remplir :
#   EDGE_NETWORK, DATA_NETWORK, TRAEFIK_CERTRESOLVER, TRAEFIK_ENTRYPOINT
#   DATABASE_URL=postgres://gathe:***@<pg-host>:5432/gathe_prod
#   AWS_S3_ENDPOINT_URL / AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / AWS_STORAGE_BUCKET_NAME
#   GATHE_IMAGE_TAG=main

# 5) Authentification au registre d'images
docker login ghcr.io -u <user> --password-stdin # PAT read:packages

# 6) DNS : app./api./cms./portail./admin. -> IP du serveur cliente (AVANT 1er start, pour l'ACME)

# 7) Post-déploiement (après la 1re mise en ligne) : seeds + AppSettings
bash /opt/gathe-finance/infra/scripts/post-deploy-seed.sh
```

C. Configuration GitHub (côté dépôt)

Type	Recette (existant)	Clienté (à poser)
Secrets	<code>VPS_HOST</code> , <code>VPS_USER</code> , <code>VPS_SSH_PRIVATE_KEY</code>	<code>CLIENT_VPS_HOST</code> , <code>CLIENT_VPS_USER</code> , <code>CLIENT_VPS_SSH_PRIVATE_KEY</code> , <code>CLIENT_VPS_DEPLOY_PATH</code>
Variables	<code>STAGING_SITE_DOMAIN</code> ...	<code>CLIENT_SITE_DOMAIN</code> , <code>CLIENT_PORTAL_DOMAIN</code> , <code>CLIENT_API_DOMAIN</code> , <code>CLIENT_ADMIN_DOMAIN</code> , <code>CLIENT_CMS_DOMAIN</code>

Les valeurs DMZ (réseaux, certresolver, `DATABASE_URL`, MinIO) ne sont **pas** de la config GitHub : elles vivent dans `infra/.env.prod` **sur le serveur**.

5. Vérification

Recette (après merge → staging)

```
curl -sL -o /dev/null -w "%{http_code}\n" https://gathe-finance.horus-lab.com/  
curl -sL -o /dev/null -w "%{http_code}\n" https://api.gathe-finance.horus-lab.com/healthz/
```

Attendu : 200 partout ; la nouvelle vitrine (texte pleine largeur, SEO, a11y, perf) est visible.

Cliente (après config + merge → main)

```
curl -sL -o /dev/null -w "%{http_code}\n" https://app.gathe-finance.com/  
curl -sL -o /dev/null -w "%{http_code}\n" https://api.gathe-finance.com/healthz/
```

Traefik de la DMZ émet les certificats (ACME) automatiquement dès que le DNS pointe et que les conteneurs portent les labels `traefik.*`.

6. Rôles & garde-fous

- Un **seul** tag d'images par serveur : recette= `staging` , cliente= `main` — jamais `latest` sur la recette une fois main = cliente.
- Le **garde-fou** de `deploy.yml` (cliente) s'ignore tant que `CLIENT_VPS_HOST` est vide → merger sur `main` reste sans risque avant la config cliente.
- Aucune migration ni changement backend n'est requis pour la mise à jour vitrine en cours (front uniquement).
- Rollback recette : re-déclencher `deploy-staging.yml` sur un `:sha-xxxx` antérieur (`workflow_dispatch`).